Funciones y Arreglos

Corporación Universitaria Empresarial
Alexander Von Humboldt

AUTOR

Lauren Victoria Lopez Quebrada
Hugo Marin

PROGRAMA ACADÉMICO

Ingeniería de Software I

ASIGNATURA

Algoritmos y Lógica Computacional

DOCENTE

Arle Morales

Armenia - Quindio

2026

NIVEL 1 - La Máquina de Una Sola Tecla

Calentamiento antes del Nivel 1

Cinco minutos, sin computador. El docente proyecta cada expresión y el equipo levanta la mano: arriba si es `true`, abajo si es `false`. Cada acierto vale **+6 XP**. Al terminar, el Piloto las escribe en la consola: la respuesta `true` o `false` es la base de todas las condiciones de hoy.

#	Expresión	Nuestra respuesta	¿Acertamos?	Qué aprendimos de esta
1	`7 > 3`	True	Si	7 es Mayor que, por lo que > da verdadero
2	`"gato" === "Gato"`	False	Si	Al comparar textos, las mayusculas y minusculas cuentan.
3	`10 <= 10`	True	Si	<= incluye la igualdad, asi que dos valores iguales dan verdadero.
4	`5 !== 5`	False	Si	Se pregunta si son distintos y 5 en este caso si es igual a "5"
5	`2 + 2 === 4`	True	Si	Primero se calcula la suma y después se compara con ===

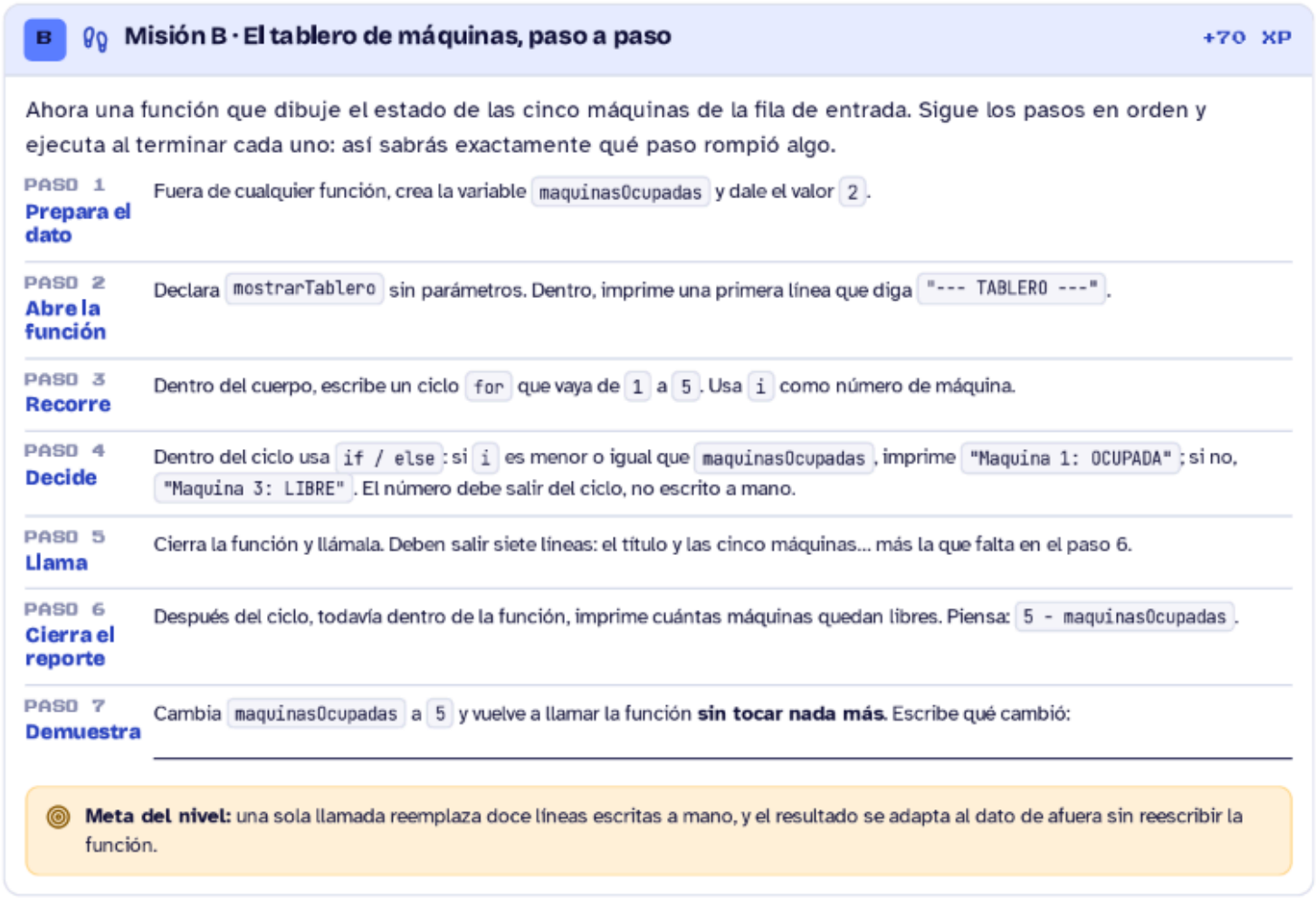
Mision A.

A · Misión A · El cartel de bienvenida — +50 XP

Escribe una función llamada `mostrarBienvenida` que imprima **tres líneas fijas**: el nombre del salón, el horario y el precio de la ficha. Después llámala **dos veces seguidas** y observa la consola.

- [x] Se declara con `function` y los paréntesis van vacíos.
- [x] Las tres líneas salen con `console.log` desde dentro del cuerpo.
- [x] Llamarla dos veces imprime el cartel dos veces, sin repetir código.
- [x] El nombre empieza por un verbo y no lleva espacios ni tildes.

Para unir texto y número usa `+`: `"Ficha: $" + 2500`. Escribe en el cuadernillo el nombre exacto que le pusiste a tu función: mostrarBienvenida



Paso 7: Al cambiar maquinasOcupadas a 5, las cinco máquinas salen “OCUPADA” y el reporte final dice "Libres: 0". La función no se modificó: cambió solo el dato de afuera, que la función lee cada vez que se ejecuta.

Control de conocimiento

Estación 6 · +40 XP

Se responde con el editor cerrado y el cuadernillo abierto. El Cronista escribe; todo el equipo discute antes de anotar. 10 XP por respuesta correcta.

Q1 Explica con tus palabras la diferencia entre escribir `mostrarTablero` y escribir `mostrarTablero()`.

Escribir mostrarTablero solo nombra la función: no se ejecuta y no pasa nada visible. Escribir mostrarTablero() la llama: los paréntesis son la orden de arranque y ejecutan el cuerpo.

Q2 Este programa se ejecuta completo. ¿Qué aparece en la consola?

```
function avisar() {
  console.log("Máquina lista");
}
console.log("Abriendo...");
```

○ `Máquina lista` ✔ `Abriendo...` ○ Las dos líneas ○ Nada

¿Por qué? Porque la función avisar() se declara pero nunca se llama, así que su console.log no se ejecuta.

Q3 Si una función tiene cuatro `console.log` en su cuerpo y la llamas tres veces, ¿cuántas líneas salen en total? Salen 12 Lineas Explica la cuenta: cada llamada ejecuta los 4 console.log, y como se llama 3 veces, 4 × 3 = 12.

Q4 El mismo bloque de doce líneas se necesita en tres momentos distintos del día. Da **dos razones** por las que conviene una función en vez de copiar y pegar el bloque tres veces.

1. El bloque se escribe una sola vez; si hay que cambiarlo, se corrige en un solo lugar y no en tres copias.
2. Cada momento del día necesita solo una línea de llamada, así que el código es más corto y hay menos riesgo de errores por copiar y pegar.

8 Auditoría con IA

+40 XP

Solo cuando tus dos misiones ya funcionen. El Auditor abre la IA, pega el prompt tal cual y reemplaza la línea entre corchetes por su código.

```
Estoy aprendiendo funciones en JavaScript. Este es mi código:

[pega aquí tu función mostrarTablero completa]

Responde en español y sin reescribirme todo el código:
1. ¿El cuerpo se ejecuta si nunca llamo la función?
2. ¿El nombre describe lo que hace? Propón otro si no.
3. Dime UN error o UNA mejora concreta, y explícame por qué.
No me des la solución final.
```

Prueba cada sugerencia en la consola antes de juzgarla. Los 40 XP se ganan por el veredicto, no por obedecer.

Sugerencia de la IA	¿La probaste?	Acertada / Discutible / Equivocada	Por qué lo decidimos así

Cazador de errores (+15 XP): si la IA afirma algo que la consola desmiente, escríbelo aquí:

SUGERENCIA IA	¿LA PROBASTE?	ACERTADA/ DISCUTIBLE EJECUTADO
1. El cuerpo de la función no se ejecuta a menos que se invoque escribiendo su nombre con paréntesis (mostrarTablero())	Si	ACERTADA

POR QUE LO DECIDIMOS ASI

Al probar comentando o borrando las llamadas de las líneas 15 y 18 en la consola, comprobamos que el texto del tablero no aparece hasta que la función es llamada explícitamente.

Lo que te llevas del Nivel 1

DECLARAR	LLAMAR	CUERPO	CONTEXTO
`function saludar() { }` guarda la receta. Por sí sola no imprime nada.	`saludar()` ejecuta el cuerpo. Puedes llamarla las veces que quieras.	Dentro de las llaves cabe todo: `console.log`, `for`, `if`, otras funciones.	Lee las variables de afuera en el momento de ejecutarse, no cuando la escribiste.

Cierre del Nivel 1 · marca con honestidad	Lo domino	Casi	Ayuda
Declaro una función y la llamo sin equivocarme de paréntesis.	○	✔	○
Uso un `for` y un `if` dentro del cuerpo de una función.	✔	○	○
Explico por qué la misma llamada puede dar resultados distintos.	✔	○	○

XP del nivel: _____ · Firma del Cronista: _____ · Rol que te toca en el Nivel 2: _____

NIVEL 2 Tu función ya hace cosas, pero siempre las mismas. En el siguiente nivel aprenderás a **pasarle datos** y a hacer que **devuelva un resultado**, para calcular el precio de cualquier paquete de fichas sin escribir una función por cada precio.

NIVEL 2 - Entrada y Salida

A · Misión A · El precio del paquete +50 XP

Escribe `precioPaquete(cantidad)`: recibe cuántas fichas quiere el cliente y **devuelve** el total a pagar aplicando la tabla de arriba. No imprimas nada dentro de la función.

- ☑ Tiene exactamente un parámetro y termina con un `return`.
- ☑ Usa una decisión para elegir entre $2 500 y $2 000 por unidad.
- ☑ Prueba con 1, 4, 5 y 12 fichas imprimiendo el resultado **desde fuera**.
- ☑ Con 5 fichas debe dar 10 000 y con 4, 10 000 también: anota por qué ese salto es raro para el cliente.

Resultado de tus cuatro pruebas: 1 → 1 * 2500 = $2500 · 4 → 4 * 2500 = $10000 · 5 → 5 * 2000 = $10000 · 12 → 12 * 2000 = $24000

MISION B

Al terminar la partida, la máquina entrega puntos. La taquilla los cambia por tickets. Vas a construir la función que hace ese cambio, con dos parámetros.

PASO 1 — Declara
Crea `puntosATickets` con **dos parámetros**: `puntos` y `esOro`. El segundo será `true` o `false`.

PASO 2 — Convierte
Dentro, calcula los tickets base: cada 100 puntos es 1 ticket. Guarda el resultado en una variable.

PASO 3 — Redondea
Los tickets no se parten. Usa `Math.floor(...)` para quedarte solo con la parte entera.

PASO 4 — Decide
Si `esOro` es `true`, duplica los tickets. Si no, déjalos igual. Un `if` basta.

PASO 5 — Devuelve
Termina con `return` entregando el número final de tickets. Nada de `console.log` dentro.

PASO 6 — Prueba
Llámala cuatro veces desde fuera e imprime cada resultado: `(250, false)`, `(250, true)`, `(99, false)` y `(1000, true)`.

Llamada	Predicción	Resultado real
`puntosATickets(250, false)`	2	2
`puntosATickets(250, true)`	4	4
`puntosATickets(99, false)`	0	0
`puntosATickets(1000, true)`	20	20

PASO 7 — Encadena
Usa el resultado de una función dentro de otra: imprime cuánto costaría comprar tantas fichas como tickets ganó el cliente oro con 1 000 puntos, llamando a `precioPaquete` con el resultado de `puntosATickets`.

Meta del nivel: el valor que sale de una función puede entrar directo en otra. Así se construye todo lo que viene después.

Control de conocimiento

- **Q1.** `b` vale **undefined**, porque solo se pasó un argumento y el segundo parámetro queda vacío. `console.log(a + b)` imprime **NaN**, ya que 10 + undefined no es un número.
- **Q2.** Opción: **solo 10.** Porque `return` termina la función en esa línea, así que el `console.log("listo")` que viene después nunca se ejecuta.
- **Q3.** A su función le falta el `return`. Imprimir muestra el valor pero no lo entrega, así que `r` queda en `undefined` y `undefined * 2` da `NaN`. Debe terminar con `return` del resultado.
- **Q4.** El parámetro es el nombre `cantidad` que se escribe al declarar `precioPaquete(cantidad)`; el argumento es el valor real que se manda al llamar, como el `12` en `precioPaquete(12)`.

NIVEL 3 - La flecha

A · Misión A · Tres traducciones +50 XP

Traduce estas tres funciones a **flecha de una sola línea**, sin llaves y sin `return`. Compruébalas llamándolas después.

para-traducir.js

```
function mitad(n) { return n / 2; }
function esCaro(precio) { return precio > 5000; }
function etiqueta(juego, piso) { return juego + " (piso " + piso + ")"; }
```

- [x] Las tres quedan guardadas en `const` y conservan su nombre.
- [x] Ninguna tiene llaves ni la palabra `return` .
- [x] `esCaro` sigue devolviendo `true` o `false` , no un texto.
- [x] Pruébalas: `mitad(9)` → 4.5 · `esCaro(2500)` → false

PASO 5 **Arma el recibo**

Con un precio de `18000` , un descuento del `25` %, el juego `"Galaxia Infinita 2"` y un saldo de `15000` , imprime cuatro líneas usando tus cuatro herramientas.

Línea del recibo	Lo que debe salir
Nombre recortado del juego	Galaxia Inf
Precio con descuento	13.500
¿Es gratis?	False
¿Le alcanza el saldo?	True

PASO 6 **Rompe y repara**

Ponle llaves a `esGratis` sin añadir `return` y ejecútala. Anota qué devuelve ahora: undefined Después arréglala.

PASO 7 **Argumenta**

De tus cuatro herramientas, ¿alguna quedaría más clara como `function` clásica? Cuál y por qué:
aplicarDescuento, porque al tener una fórmula matemática un poco más larga con varias operaciones, una estructura clásica con la palabra return facilita la lectura visual del cálculo

Meta del nivel: escribir en una línea lo que antes ocupaba cinco, sin que nadie del equipo tenga que adivinar qué hace.

Control de conocimiento

- **Q1.** Imprime **undefined**. Para que dé 12 hay que quitar las llaves (`const triple = n => n * 3;`) o escribir `return` dentro de ellas.
- **Q2.** `const sumar = (a, b) => a + b;`
- **Q3.** En `a = (x) => x + 1` los paréntesis sobran, porque con un solo parámetro son opcionales. En `b = (x, y) => x + y` son obligatorios, porque con dos o más parámetros hay que ponerlos.
- **Q4.** Prefiero `function` para `precioPaquete` , porque tiene un `if/else` y varias líneas, y un bloque largo con decisiones se lee mejor como declaración clásica.

NIVEL 4 - La caja de datos

A · Misión A · El catálogo en pantalla

+50 XP

Crea un arreglo `catalogo` con **seis** nombres de juego inventados por tu equipo y responde con código, no a mano.

- [x] Imprime el primer juego y el último, este segundo usando `length` (no escribas el 5).
- [x] Imprime cuántos juegos hay en total.
- [x] Reemplaza el juego de la posición 2 por `"EN MANTENIMIENTO"` y vuelve a imprimir el arreglo completo.
- [x] Imprime `catalogo[10]` y anota qué sale: _____

Cuidado con la palabra "posición 2": en la vida diaria es el segundo; en un arreglo, `catalogo[2]` es el **tercero**. Decide en equipo cuál usar y déjalo escrito.

aunque el índice empiece en 0.

PASO 3 — Acumula

Antes del ciclo crea `total = 0`. Recórrelo con `for…of` y súmale cada puntaje.

PASO 4 — Cuenta

En el mismo recorrido, lleva un `clasificados` que suba `+1` cada vez que un puntaje sea **mayor o igual** a 500. Ojo con el 500 exacto.

PASO 5 — Compara

También en el mismo recorrido, busca el puntaje `mayor`. Arráncalo con `puntajes[0]`, nunca con 0.

PASO 6 — Reporta

Después del ciclo imprime el resumen y anota aquí lo que salió:

Dato del tablero	Tu resultado
Total de puntos de la ronda	4375
Cuántos clasificaron (≥ 500)	5
Puntaje más alto	1200
Promedio con un decimal	546.9

PASO 7 — Encuentra el puesto

Guarda también en qué **índice** estaba el puntaje mayor y conviértelo al número de partida que vería el público. Índice: 7 Partida: 8

Meta del nivel: un solo recorrido puede calcular varias cosas a la vez. Cuatro respuestas, ocho datos, un `for`.

Paso 7: el índice del puntaje mayor es **7** y el número de partida es **8**.

Control de conocimiento

- **Q1.** `a[1]` vale **20**, `a.length` vale **4** y `a[a.length]` da **undefined**.
- **Q2.** Imprime **8**. Daría un resultado equivocado con un arreglo de números negativos, como `[-5, -8, -2]`: imprimiría 0, que no está en la lista, porque `m` arranca en 0.
- **Q3.** El `for` clásico conviene cuando necesito el índice, y `for...of` cuando solo me importa el valor de cada elemento.
- **Q4.** Hay que cambiar `<=` por `<`. El último índice es `length - 1`, y con `<=` el ciclo da una vuelta de más y lee una casilla que no existe, por eso sale `undefined`.

NIVEL 5 - Los que dejan huella

Misión A · La fila del torneo +50 XP

Crea `const fila = ["Ana", "Beto", "Caro", "Dani"];` y simula media hora de torneo. **Predice antes de ejecutar** cada paso y anótalo.

- [x] Llegan dos jugadores nuevos al final de la fila.
- [x] Se libera una máquina: entra el primero de la fila. Guarda su nombre en una variable e imprímelo.
- [x] Llega alguien con pase VIP y se pone de primero.
- [x] El último de la fila se cansa y se va.

Estado final de la fila: ["Santi", "Beto", "Caro", "Dani", "Eva"] · Cuántos quedan: 5

B · Misión B · Mantenimiento del catálogo, paso a paso +70 XP

Trabaja sobre `["Pinball", "Galaxia", "Karts", "Zombis", "Tetris", "Aereo"]`. Imprime el arreglo completo después de cada paso: si no lo ves, no lo sabes.

PASO 1 — Copia de seguridad

Antes de tocar nada, guarda el tamaño original en `const tamanoInicial = catalogo.length;`. Lo vas a necesitar al final.

PASO 2 — Retira

El técnico reporta averiados `"Karts"` y `"Zombis"`, que están juntos. Sácalos con **un solo** `splice` y guarda lo que devuelve en `retirados`.

PASO 3 — Verifica

Imprime `retirados` y el catálogo. ¿Cuántos elementos tiene ahora? **4**

PASO 4 — Reemplaza

Llega una máquina nueva, `"Laberinto"`, que debe quedar exactamente donde estaba `"Karts"`. Insértala con `splice` sin borrar nada.

PASO 5 — Prioriza

El dueño quiere `"Pinball"` de último en el tablero. Sácalo del principio y vuelve a meterlo al final usando dos métodos distintos del nivel.

PASO 6 — Invierte

El LED se va a leer de derecha a izquierda esta noche. Da vuelta el catálogo y anota el resultado:

Momento	Contenido del catálogo	length
Tras retirar averiados	["Pinball", "Galaxia", "Tetris", "Aereo"]	4
Tras insertar Laberinto	["Pinball", "Galaxia", "Laberinto", "Tetris", "Aereo"]	5
Tras mover Pinball	["Galaxia", "Laberinto", "Tetris", "Aereo", "Pinball"]	5
Tras invertir	["Pinball", "Aereo", "Tetris", "Laberinto", "Galaxia"]	5

PASO 7 — Demuestra la huella

Compara el `length` final con `tamanoInicial` e imprime si el catálogo creció, se redujo o quedó igual. Luego responde: ¿podrías recuperar el orden original sin volver a escribirlo? **Comparación:** El catálogo **se redujo** (pasó de 6 a 5 elementos).

- **Sí**, aplicando nuevamente reverse() y desandando los movimientos realizados con los métodos de arreglos.

Nivel 6 - La fabrica

A · Misión A · Los precios del festival +50 XP

Parte de `const precios = [2500, 18000, 9000, 4500, 12000];` y genera con `map` un arreglo nuevo llamado `preciosFestival` con el 20 % ya descontado.

- [x] Usas una flecha de una sola línea, sin llaves y sin `return`.
- [x] Imprimes los dos arreglos y compruebas que `precios` no cambió.
- [x] Imprimes los dos `length` y confirmas que son iguales.
- [x] Añades un segundo `map` que convierta cada precio del festival en el texto `"$ 2000"`.

B **Misión B · Las etiquetas del tablero, paso a paso** +70 XP

El LED de entrada recibe una lista de textos ya listos para mostrar. Tú la vas a fabricar a partir de dos arreglos de datos crudos.

PASO 1 **Los datos** — Crea `const juegos = ["pinball", "karts", "zombis", "laberinto"];` y `const partidas = [320, 980, 75, 640];`. Cada juego corresponde al puntaje de su misma posición.

PASO 2 **Mayúsculas** — Con `map`, crea `nombresLed` poniendo cada nombre en mayúsculas. Usa `.toUpperCase()`.

PASO 3 **Puesto** — Con otro `map`, crea `conPuesto` con el formato `"1. PINBALL"`. Necesitas el índice: acuérdate del segundo parámetro.

PASO 4 **Tickets** — Con un tercer `map` sobre `partidas`, crea `tickets` dividiendo entre 100 y quedándote con la parte entera (`Math.floor`).

PASO 5 **Une los** — Con un `map` sobre `juegos` usando el índice, arma `etiquetas` con el formato `"PINBALL — 3 tickets"`, tomando el ticket de

NIVEL 7 - El Colador

Paso 7 (respuesta):
Solo con map no se puede: map devuelve siempre el mismo tamaño, no descarta elementos.
Solo con filter tampoco: filter no cambia los elementos, no los pone en mayúsculas.

NIVEL 8 - El detector

A **Misión A · ¿Se puede abrir el salón?** +50 XP

Parte de `const revision = [true, true, true, false, true, true];`, donde cada posición es una máquina.

- [ ] Responde con `every` si se puede abrir el salón e imprime el resultado.
- [ ] Responde con `some` si hay que llamar al técnico. Cuidado: la pregunta busca los `false`.
- [ ] Mete las dos respuestas dentro de un `if / else` que imprima `"ABRIR"` o `"LLAMAR TECNICO"`.
- [ ] Cambia el `false` por `true` y vuelve a ejecutar sin tocar nada más.

Predice antes de ejecutar. El Cronista llena la columna del medio con el editor cerrado.

Pregunta	Creemos	Sale	Método que usamos
¿Se puede abrir el salón?			
¿Hay que llamar al técnico?			
Con todas en `true`, ¿qué imprime?			

PASO 6 **El caso raro** — Ejecuta tu semáforo con `const ronda = [];` sin cambiar nada más. Anota qué color sale: _____ ¿Tiene sentido para el torneo? _____

PASO 7 **Compara herramientas** — Resuelve la pregunta 1 otra vez, ahora con `filter` y `length`. ¿Da lo mismo? _____ ¿Cuál de las dos versiones prefieres y por qué?

_____

Paso 6 (con ronda = []):
hayEstrella = false, rondaValida = true (every sobre vacío), todoEnOrden = false
-> sale AMARILLO. No tiene mucho sentido: una ronda sin partidas no debería ser "válida

Q1: some(x > 10) -> true | every(x > 10) -> false | filter(x > 10) -> [16]
// Q2: false y true. Sobre [] some no encontró ninguno (false);
// every no encontró ninguno que falle (true).
// Q3: !averiadas.some(a => a === true)
// Q4: some se detiene en cuanto encuentra el primero que cumple;
// filter recorre las diez mil partidas y además construye un arreglo que no necesitas.

Nivel 9 - La torre de control

A · Misión A · El ranking correcto +50 XP

Parte de `const llegada = [320, 1200, 75, 980, 510];`, que es el orden en que se jugaron las partidas.

- [x] Ordena una **copia** sin comparador e imprime el desastre resultante.
- [x] Ordena otra copia de mayor a menor con el comparador correcto.
- [x] Imprime `llegada` al final: debe seguir en el orden original.
- [x] Ahora ordena `llegada` sin copia y comprueba que ya no se puede recuperar.

B · Misión B · La tabla de campeones, paso a paso +70 XP

Ahora con objetos, que es como se guardan los datos de verdad. Un objeto por jugador, con su nombre y sus puntos.

PASO 1 **Los datos** — Crea el arreglo `jugadores` con cinco objetos de la forma `{ nombre: "Ana", pts: 320 }`. Inventa los nombres; usa estos puntos: 320, 1200, 75, 980, 510.

PASO 2 **Comprueba el acceso** — Imprime el nombre del tercer jugador del arreglo. Recuerda el índice del Nivel 4 y el punto para la propiedad.

PASO 3 **Ranking** — Ordena una **copia** de mayor a menor por `pts`. Guárdala en `ranking` e imprime el nombre del campeón.

PASO 4 **Folleto** — Ordena otra copia por `nombre` con `localeCompare`. Guárdala en `alfabetico`.

NIVEL 10 - La Última Vuelta

A · Misión A · El recorrido que imprime +50 XP

Parte de `const finalistas = ["Ana", "Beto", "Caro", "Dani"];` y practica el recorrido antes de encadenar.

- ☑ Imprime con `forEach` una línea por finalista con el formato `"Puesto 1: ANA"` .
- ☑ Guarda en una variable lo que devuelve ese `forEach` e imprímela. Anota qué sale: _____
- ☑ Haz lo mismo con `map` en vez de `forEach` y compara las dos variables.
- ☑ Explica en una línea por qué `map` no sirve aquí y `forEach` sí.

B · Misión B · El tablero final, paso a paso +70 XP

El reto de cierre. Construye la cadena eslabón por eslabón, ejecutando después de cada paso.

PASO 1 **Los datos**	Crea el arreglo `partidas` con seis objetos `{ nombre: "Ana", pts: 320 }`. Usa estos puntos: 320, 1200, 75, 980, 510, 440.
PASO 2 **Filtra**	Encadena un `filter` que deje solo a los que llegan a 500. Imprime cuántos quedaron antes de seguir.
PASO 3 **Ordena**	Sobre ese resultado, encadena un `sort` de mayor a menor por `pts`. Piensa si aquí hace falta la copia con `[...]` y justifícalo: _____
PASO 4 **Transforma**	Encadena un `map` que convierta cada objeto en un texto: `"1. BETO — 12 tickets"`. Necesitas el índice, las mayúsculas y la división entre 100.
PASO 5	Cierra la cadena con un `forEach` que muestre cada línea. Escribe la cadena completa en líneas separadas, un eslabón por línea.

Lo que te llevas del Nivel 10

FOREACH	CADENA	ORDEN	ORIGINAL
Recorre para hacer algo. Devuelve `undefined` siempre.	Cada eslabón trabaja sobre lo que devolvió el anterior.	Filtrar antes de transformar. `forEach` siempre al final.	Solo `sort` lo toca. Con `[...]` también queda a salvo.

Cierre del Nivel 10 · marca con honestidad	Lo domino	Casi	Ayuda
Uso `forEach` para imprimir y `map` para construir.	○	✔	○
Encadeno varios métodos y sé leer la cadena de arriba abajo.	○	✔	○
Justifico el orden de los eslabones de mi cadena.	○	○	✔